

URL Shortener — Project Submission

Author: parimala-1611 **Date:** 2026-07-08 **Repositories:**

- Backend (Spring Boot / Java): <https://github.com/parimala-1611/UrlShortner>
 - Frontend (React / TypeScript): <https://github.com/parimala-1611/urlshortner-frontend>
-

1. Overview

This submission is a full-stack URL shortener — a Spring Boot/Java backend and a React/TypeScript frontend — built alongside a **real, runnable SDLC orchestration engine** that governs how every feature in both repositories moves from requirements through to a human-approved release. The assignment explicitly called out agentic orchestration as the top evaluation criterion and "critical differentiator," so the core engineering decision made early in this project was to build a real engine (gates that actually block, retries that actually retry, rollbacks that actually revert) rather than documenting an orchestration *policy* in prose and treating ordinary development as an implicit substitute for it.

That engine (`orchestrator/` , Python, standard library only) was built once, against the backend, then **adapted and reused** against the frontend — proving it isn't tied to one language or one repo. Across both repositories it drove **five real feature scenarios** end to end, each with its own requirements/design artifacts, a real automated test run, real `git diff` -based compliance checks, and a human-approval checkpoint that halted the pipeline until an explicit, audited decision was made.

2. What Was Built

2.1 Backend — `UrlShortner`

Spring Boot 3.3.5 / Java 21 REST API, PostgreSQL + Flyway migrations, no authentication (every endpoint is public/anonymous).

Feature	Behavior
Shorten a URL	Normalizes the URL (lowercase scheme/host, strip default ports/trailing slash/fragment) and dedupes — the same URL always returns the same short code.
Custom alias	Optional 6-12 character alias; if taken, silently falls back to the generated code instead of erroring.
Redirect	<code>GET /{shortCode}</code> → <code>302</code> to the original URL, increments the click counter and records a click event.
Expiry & retention	Configurable default expiry (365 days) if none is supplied; expired links return <code>410 Gone</code> ; a scheduled job hard-deletes links well past expiry (90 days).
Strict URL validation	Only real <code>http / https</code> URLs — rejects other schemes, gibberish hosts, and filename-shaped hosts (e.g. <code>malware.exe</code>).
QR code	<code>GET /api/urls/{shortCode}/qr</code> — PNG QR code encoding the short link (ZXing), works even for expired links.
Analytics	<code>GET /api/urls/{shortCode}/analytics</code> — total clicks, daily click-count breakdown, top referrers (capped at 10), aggregated in Java (not SQL) so the logic is unit-testable without a database.
CORS	Disabled by default (secure by default); one <code>application.yml</code> property (<code>app.cors.allowed-origins</code>) enables specific origins.

Tech: Java 21, Spring Boot (Web, Data JPA, Validation), PostgreSQL, Flyway, Hibernate, Testcontainers (integration tests), Docker/Docker Compose, GitHub Actions CI.

2.2 Frontend — `urlshortner-frontend`

React 19 + TypeScript, built with Vite, styled with Tailwind CSS 4.

Page	Behavior
Shorten (/)	Submit a URL with an optional custom alias and expiry. Client-side validates the URL (scheme, filename-shaped hosts) and alias (6-12 alphanumeric) before calling the API. If the requested alias wasn't honored (silently substituted by the backend), shows a distinct notice explaining what happened instead of silently displaying the substitute link. Keeps a local link history (<code>localStorage</code>).
Stats (/stats/:shortCode)	Fetches stats, QR code, and analytics independently (not <code>Promise.all</code>) so one endpoint 404ing doesn't hide the other two. Renders the QR code as an image (with a graceful fallback on load failure), and analytics as total clicks + a hand-rolled inline-SVG daily-clicks chart (zero-filling the days the API omits) + a top-referrers list.

Tech: React 19, TypeScript, Vite, React Router, Tailwind CSS 4, Vitest, React Testing Library, oxlint.

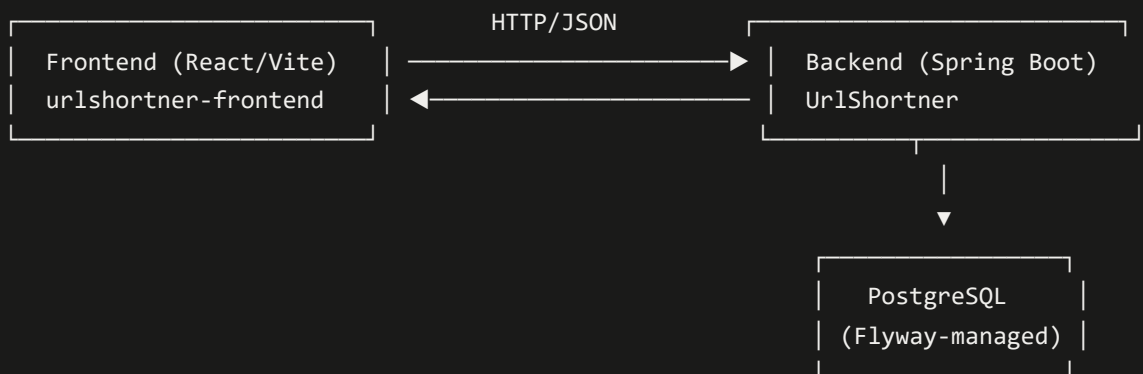
2.3 SDLC Orchestration Engine — orchestrator/

A DAG-based pipeline engine, present in both repositories (the frontend's copy is adapted to run `npm test /Vitest` instead of `mvnw test /JUnit` — everything else is identical):

Component	Responsibility
<code>pipeline.py</code>	Loads a pipeline spec (JSON): stages, dependencies, gates. Validates the dependency graph (unknown deps, cycles) at load time.
<code>engine.py</code>	The scheduler: entry/exit gates, concurrent execution of independent stages (<code>ThreadPoolExecutor</code>), bounded retry with backoff, rollback (<code>git reset --hard</code> to a captured pre-stage SHA), human-approval halts, bounded dynamic re-planning, safe-stop on unrecoverable failure.
<code>audit.py</code>	Append-only JSON-lines audit log per run, plus a human-readable summary.
<code>metrics.py</code>	Computes success rate, retry/rollback frequency, MTTR, and end-to-end latency from real audit logs.
<code>guardrails.py</code>	Policy checks against the real <code>git diff</code> : no secret-shaped strings, no unapproved protected-file changes, a change-size bound.
<code>stages/</code>	Generic stage runners: <code>requirements</code> , <code>design</code> (artifact existence/shape checks), <code>implementation / docs</code> (real <code>git diff</code> checks), <code>test</code> (really runs the project's test suite and gates on the process exit code), <code>compliance</code> , <code>release_readiness</code> (writes a release manifest once approved).
<code>pipelines/*.json</code>	One spec per scenario — stage graph, dependencies, and each stage's retry/rollback/approval policy.
<code>artifacts/</code>	Requirements and design write-ups (real authored content, tracked in git) and release manifests.
<code>runs/</code>	Per-run audit logs and state — generated output, gitignored.

Design principle: cognitive stages (requirements, design) are authored by the AI agent and *gated* by the engine (artifact must exist, be non-empty, and — when a request is flagged ambiguous — contain an explicit `## Assumptions` section); mechanical stages (test, docs, compliance) are *genuinely automated* via `subprocess` calls to the real build tooling and real `git diff` , so a pass/fail signal is always a verified fact, never an assertion.

3. Architecture



Both repos additionally carry:

```

orchestrator/ (Python, stdlib only) – DAG engine, gates, audit, metrics
requirements → design → implementation → (test || docs) → compliance
→ release_readiness (approval)
  
```

4. SDLC Orchestration — The Five Scenarios

Every scenario below was actually run via `python orchestrator/run.py start ...` against the real repo — the audit logs, test results, and `git diff`s referenced are real, not narrated.

#	Repo	Scenario	What it demonstrates
1	Backend	Greenfield — QR code generation	Brand-new capability, no existing code touched. Real failure loop: Windows <code>mvnw.cmd</code> subprocess resolution bug, missing <code>JAVA_HOME</code> , and the <code>compliance</code> guardrail correctly blocking <code>pom.xml</code> (a protected file) for a new dependency — each found and fixed for real, not staged.
2	Backend	Brownfield — CORS configuration	Enhancement to existing code, chosen because it was <i>already documented</i> as a known gap — a genuine before/after. Completed cleanly on the first attempt once the greenfield bugs were fixed.
3	Backend	Ambiguous — "Add analytics"	Deliberately vague input. The <code>requirements</code> stage itself is approval-gated (not just release) — a human signs off on the <i>interpreted scope</i> before any design/implementation work happens. Halted twice for real: once after requirements, once before release.
4	Frontend	Custom alias support	Adapting the engine to a second language/stack (npm/Vitest instead of Maven/JUnit) and driving a real UI feature through it. Caught a real bug in the engine itself (below).
5	Frontend	Link details (QR + analytics)	A second frontend feature, consuming backend endpoints the UI hadn't used yet, run clean on the first attempt using the fixed engine. Chart component built following the <code>dataviz</code> design-system-agnostic method (color, mark, and accessibility conventions), not ad hoc.

Aggregate evidence (backend, `orchestrator/artifacts/aggregate_metrics_report.json`): 3 runs, 21/21 stages passed, 100% success rate on final recorded runs. Frontend runs (scenarios 4-5): 6/6 automated stages passed each, 1 human approval each, 0 rollbacks on the final recorded attempt.

5. Real Bugs Found and Fixed

Running the engine for real — not just reasoning about its design — surfaced genuine defects, in both repositories:

Bug	Where	Root cause	Fix
<code>mvnw.cmd</code> subprocess resolution failure	Backend, greenfield scenario	Windows shell resolution for the Maven wrapper under Python's <code>subprocess</code>	Resolve <code>mvnw</code> via an absolute path, invoke with <code>shell=True</code> on Windows
Missing <code>JAVA_HOME</code>	Backend, greenfield scenario	Not set at the OS level in the dev environment	Auto-discover a JDK install and set it for the subprocess environment only
Rollback ate an uncommitted orchestrator fix	Backend, engine development	<code>git reset --hard</code> resets the <i>whole</i> working tree, not just the code being gated — a fix to <code>test.py</code> left uncommitted was wiped by a rollback triggered by an unrelated stage failure	Documented; established practice of committing orchestrator fixes immediately, never leaving them uncommitted across a run
<code>git diff <base_ref></code> blind to brand-new files	Frontend, custom-alias scenario	<code>git diff</code> only ever compares paths already present in the index — a file that was never <code>git add</code> -ed at all (not even staged) is structurally invisible to it, regardless of how the diff is invoked. This silently undercounted a real feature's change set (5 files reported vs. 12 actual), weakening the <code>implementation / docs / compliance</code> gates without any visible error.	<code>git add -N</code> (intent-to-add — records the path without staging content) once at engine start, so every downstream git-diff-based gate sees the real change set. Very likely present in the backend's original engine too, since it predates this fix.

Each of these is documented in place (`docs/SCENARIOS.md` , `docs/TESTING_AND_LIMITATIONS.md` , `docs/ORCHESTRATION_ARCHITECTURE.md`) rather than smoothed over — the point of running a real engine against real changes is that it *can* surface real defects, and it did.

6. Testing & Quality

Suite	Count	Notes
Backend unit/slice tests	80 (outside Testcontainers-dependent classes)	Mockito unit tests for pure logic (<code>UrlNormalizer</code> , <code>Base62Encoder</code> , <code>ShortCodeGenerator</code> , <code>ShortUrlService</code> , <code>AnalyticsService</code>); <code>@WebMvcTest</code> slice tests for controllers and CORS (real <code>Origin</code> /preflight headers through <code>MockMvc</code> , asserting on the real filter's response).
Backend integration tests	3 classes (Testcontainers, real Postgres)	<code>ShortUrlEndToEndTest</code> , <code>ShortUrlConcurrencyStressTest</code> (50 concurrent threads proving dedup/code-generation correctness under contention), <code>UrlShortenerApplicationTests</code> . Compile cleanly; designed to pass on CI (<code>ubuntu-latest</code>) — blocked locally by a Testcontainers/Docker Desktop API version mismatch on this dev machine, a documented environment limitation, not a defect.
Frontend tests	23	Vitest + React Testing Library: validation logic (table-driven), <code>ShortenPage</code> (client-side gating, fallback notice), <code>StatsPage</code> (independent QR/analytics fetch and graceful degradation), <code>DailyClicksChart</code> (zero-fill, per-bar tooltip). API calls mocked at the client boundary — no network required.
Orchestration engine tests	19	Real control-flow, not mocked: retry exhaustion, rollback against a real throwaway git repo, approval pause/resume, bounded re-planning, guardrail checks against a real <code>git diff</code> . Shared between both repos' engine copies.

CI: GitHub Actions on the backend repo (`.github/workflows/ci.yml` , `docker-publish.yml`) builds, tests, and publishes a Docker image to GHCR and Docker Hub on push.

7. How to Run

Backend:

```
docker compose up -d      # local Postgres
./mvnw spring-boot:run    # http://localhost:8080
./mvnw test
```

Frontend:

```
npm install
npm run dev                # http://localhost:5173, proxies /api to :8080
npm test
```

Orchestration engine (either repo):

```
python orchestrator/run.py start --pipeline orchestrator/pipelines/<scenario>.json --run-id <id>
python orchestrator/run.py approve my-run <stage> --pipeline <same-file> --approver "<name>"
python orchestrator/run.py report --all
python -m unittest discover -s orchestrator/tests -t .
```

Full details: `docs/SETUP.md` in each repository.

8. Known Limitations

- No authentication, rate limiting, or listing endpoint on the backend (out of scope, documented, not overlooked).
- Cognitive pipeline stages (requirements/design) are gated and audited by the engine but not autonomously *generated* by an in-process LLM call — they were authored interactively by the AI agent and submitted for the engine to gate, an accurate description of how this was actually built.
- CLI-based approval (`pending_approval.json` + a documented command), not a web UI.
- Bounded re-planning depth (`MAX_REPLANS_PER_STAGE = 1`) — guarantees termination, would need revisiting for a scenario needing more than one round-trip.
- Backend Testcontainers-dependent tests don't run locally on this dev machine (Docker Desktop API version mismatch); designed for and expected to pass on CI.
- The pipeline specs are demonstration-specific (exact expected file paths per feature); a production version would template/generate specs per change.

9. Commit Timeline (most recent first)

Frontend:

```
56418aa Document the two frontend orchestration scenarios
171f84a Add QR code and analytics to the link details page
a3f4ecd Add custom alias support with fallback notice + client-side validation
6648f62 orchestrator: fix implementation/docs/compliance gates missing brand-new files
f369ea4 Adopt SDLC orchestration engine, add Vitest tooling
bd71eb1 Docker files creation
0232696 UrlShortner frontend code
2f87671 Initial commit
```

Backend:

```
3edab40 Added apiflow.md file
ef30da3 Add remaining assignment deliverable docs: architecture, scenarios, setup, testing,
7d8f203 Add aggregate reliability metrics report across all 3 scenario runs
27661ba Add ambiguous pipeline spec and release manifest for the analytics scenario
8f8d077 Add analytics: daily click counts + top referrers (ambiguous scenario)
b4c1710 Add brownfield pipeline spec and release manifest for the CORS scenario
ed2883d Add configurable CORS support (brownfield scenario)
f53af4c Add greenfield pipeline spec and release manifest for the QR code scenario
0bc6f9b Fix engine: record 'enter' only after approval gate passes so an approval pause/res
479bd42 Fix test-stage runner: resolve mvnw via absolute path and auto-discover JAVA_HOME :
e141ae1 Add QR code generation endpoint (greenfield scenario)
5cea082 Add agentic SDLC orchestration engine (DAG scheduler, gates, retry/rollback, audit
d380f17 Added test cases
f003e4f Register UrlNormalizer, Base62Encoder, and ShortCodeGenerator as Spring beans
cb6e863 Implement Phase 2 core features: shorten, redirect, expiry, click count
7c2153b Add ShortUrl unit tests and exclude bootstrap class from coverage
d04058e Mark mvnw as executable
a8f1c65 Push images to Docker Hub in addition to GHCR
41d0af5 Add Phase 1 scaffold: schema, Spring Boot skeleton, and CI/CD pipeline
c5d391c Initial commit
```

10. Conclusion

Both the backend and frontend prototypes are feature-complete against the documented scope, but the actual differentiating work — per the assignment's own stated evaluation priority — is the orchestration layer: a real engine, reused across two different languages and stacks, that drove five independent feature scenarios through genuine gates, retries, a rollback, human-approval checkpoints, and audit logging, and that found and fixed real defects in itself along the way. Every claim in this document is backed by a real, re-runnable artifact: a test suite, an audit log, a `git diff`, or a commit — see the linked docs in each repository for the underlying evidence.